
Chapitre 6 : Le Langage de Manipulation de Données (LMD)

I. Introduction

Ce chapitre décrit l'aspect LMD (langage de manipulation des données) de MySQL. Nous verrons que SQL propose trois instructions pour manipuler des données :

- _ L'insertion d'enregistrements : INSERT ;
- _ La modification de données : UPDATE ;
- _ La suppression d'enregistrements : DELETE (et TRUNCATE).

II. Insertion d'enregistrements (INSERT)

Pour pouvoir insérer des enregistrements dans une table, il faut que vous ayez reçu le privilège INSERT. Il existe plusieurs possibilités d'insertion : l'insertion monoligne qui ajoute un enregistrement par instruction et l'insertion multiligne qui insère plusieurs enregistrements par une requête.

La syntaxe simplifiée de l'instruction INSERT monoligne est la suivante :

```
INSERT [INTO] [nomBase.] { nomTable | nomVue } [(nomColonne,...)]  
VALUES ({expression | DEFAULT},...),(...),...  
[ON DUPLICATE KEY UPDATE nomColonne = expression,...]
```

ON DUPLICATE KEY UPDATE permet de mettre à jour l'enregistrement présent dans la table, qui a déclenché l'erreur de doublon (dans le cas d'un index UNIQUE ou d'une clé primaire). Dans ce cas le nouvel enregistrement n'est pas inséré, seul l'ancien est mis à jour.

1. Renseigner toutes les colonnes

Ajoutons trois lignes dans la table Compagnie en alimentant toutes les colonnes de la table par des valeurs. La deuxième insertion utilise le mot-clé DEFAULT pour affecter explicitement la valeur par défaut à la colonne ville. La troisième insertion attribue explicitement la valeur NULL à la colonne nrue.

Tableau 1 Insertions

Instruction SQL	Commentaires
INSERT INTO Compagnie VALUES ('SING', 7, 'Camparols', 'Singapour', 'Singapore AL');	Toutes les valeurs sont renseignées dans l'ordre de la structure de la table.
INSERT INTO Compagnie VALUES ('AC', 10, 'Gambetta', DEFAULT , 'Air France');	DEFAULT explicite.
INSERT INTO Compagnie VALUES ('AN1', NULL , 'Hoche', 'Blagnac', 'Air Null');	NULL explicite.

2. Renseigner certaines colonnes

Insérons deux lignes dans la table Compagnie en ne précisant pas toutes les colonnes. La première insertion affecte implicitement la valeur par défaut à la colonne ville. La deuxième donne implicitement la valeur NULL à la colonne nrue.

Tableau 2 Insertions de certaines colonnes

Instruction SQL	Commentaires
INSERT INTO Compagnie(comp, nrue, rue, nomComp) VALUES ('AF', 8, 'Champs Elysées', 'Castanet Air');	DEFAULT implicite.
INSERT INTO Compagnie(comp, rue, ville, nomComp) VALUES ('AN2', 'Foch', 'Blagnac', 'Air Nul2');	NULL sur nrue implicite.

La table Compagnie contient à présent les lignes suivantes :

Figure 1 Table après les insertions

Compagnie

Valeur NULL

Valeur par défaut

comp	nrue	rue	ville	nomComp
SING	7	Camparols	Singapour	Singapore AL
AF	10	Gambetta	Paris	Air France
AN1		Hoche	Blagnac	Air Nul1
AC	8	Champs Elysées	Paris	Castanet Air
AN2		Foch	Blagnac	Air Nul2

3. Plusieurs enregistrements

Le script suivant ajoute trois nouvelles compagnies en une seule instruction INSERT.

```
INSERT INTO Compagnie VALUES
```

```
('LUFT',9,'Salas','Munich','Luftansa'),
```

```
('QUAN',1,'Kangouroo','Sydney','Quantas'),
```

```
('SNCM',3,'P. Paoli','Bastia','Corse Air');
```

- **Ne pas respecter des contraintes**

Insérons des enregistrements dans la table Pilote, qui ne respectent pas des contraintes. Le tableau suivant décrit les messages renvoyés pour chaque erreur (le nom de la contrainte apparaît dans chaque message, les valeurs erronées sont notées en gras).

La première erreur vient de la clé primaire, la seconde de l'unicité du nom. La troisième erreur signifie que la clé étrangère référence une clé primaire absente (ici une compagnie inexistante). La dernière concerne la contrainte en ligne NOT NULL.

Tableau 3 Insertions

Insertions vérifiant les contraintes	Insertions ne vérifiant pas les contraintes
INSERT INTO Pilote VALUES ('PL-1', 'Louise Ente', 450, 'AF');	mysql> INSERT INTO Pilote VALUES ('PL-1', 'Amélie Sulpice', 100, 'AF'); ERROR 1062 (23000): Duplicate entry 'PL-1' for key 1
INSERT INTO Pilote VALUES ('PL-2', 'Jules Ente ', 900, 'AF');	mysql> INSERT INTO Pilote VALUES ('PL-4', 'Louise Ente', 450, 'AF'); ERROR 1062 (23000): Duplicate entry 'Louise Ente' for key 2
INSERT INTO Pilote VALUES ('PL-3', 'Paul Soutou', 1000, 'SING');	mysql> INSERT INTO Pilote VALUES ('PL-5', 'Thomas Sulpice', 500, 'TOTO'); ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('bdsoutou/pilote', CONSTRAINT 'fk_Pil_compa_Comp' FOREIGN KEY ('compa') REFERENCES `compagnie` (`comp`)) mysql> INSERT INTO Pilote VALUES ('PL-6', NULL, 100, 'AF'); ERROR 1048 (23000): Column 'nom' cannot be null

❖ Données binaires

Le type BIT permet de manipuler des suites variables de bits. Des fonctions sont disponibles pour programmer le « ET », le « OU » exclusif ou inclusif, etc. La table suivante contient deux colonnes de ce type. Notez l'utilisation du préfixe « b » pour initialiser un tel type.

```
CREATE TABLE Registres (nom CHAR(5),numero BIT(2),adresse BIT(16));
INSERT INTO Registres VALUES ('COM2', b'10',b'0000010011110111');
```

❖ Énumérations

Le type ENUM est considéré comme une liste de chaînes de caractères. Toute valeur d'une colonne de ce type devra appartenir à cette liste établie lors de la création de la table. Supposons qu'on recense quatre types possibles de diplômes ('BTS', 'DUT', 'Licence' et 'INSA') pour chaque étudiant. On ne stocke qu'un seul diplôme par étudiant.

Figure 2 Table avec une colonne ENUM

UnCursus			
num	nom	{diplome}	
E1	F. Brouard	BTS	
E2	F. Degrelle	Licence	

Le script de la création de la table et des insertions est le suivant. Notez que les parenthèses sont optionnelles pour désigner la colonne ENUM.

Tableau 4 Insertions avec un ENUM

Création	Insertions (deux bonnes une illicite)
<pre>CREATE TABLE UnCursus (num CHAR(4), nom CHAR(15), diplome ENUM ('BTS','DUT','Licence','INSA'), CONSTRAINT pk_Cusus PRIMARY KEY(num));</pre>	<pre>mysql> INSERT INTO UnCursus VALUES ('E1', 'F. Brouard', ('BTS')); mysql> INSERT INTO UnCursus VALUES ('E2', 'F. Degrelle', 'Licence'); mysql> INSERT INTO UnCursus VALUES ('E3', 'Bug', ('MathSup')); ERROR 1265 (01000): Data truncated for column 'diplome' at row 1</pre>

Le type **SET** permet de comparer une liste à une combinaison de valeurs permises à partir d'un ensemble de référence (chaînes de caractères). Supposons qu'on désire stocker plusieurs diplômes par étudiant.

Figure 3 Table avec une colonne SET

Cursus			SET
num	nom	{diplomes}	BTS, DUT, Licence INSA
E1	F. Brouard	BTS, Licence	
E2	F. Degrelle	Licence, INSA, DUT	

Le script de la création de la table et des insertions est le suivant (même remarque pour les parenthèses).

Tableau 5 Insertions avec un SET

Création	Insertions (deux bonnes une illicite)
<pre>CREATE TABLE Cursus (num CHAR(4), nom CHAR(15), diplomes SET ('BTS','DUT','Licence','INSA'), CONSTRAINT pk_Cusus PRIMARY KEY(num));</pre>	<pre>mysql> INSERT INTO Cursus VALUES ('E1', 'F. Brouard', ('BTS',Licence)); mysql> INSERT INTO Cursus VALUES ('E2', 'F. Degrelle', 'Licence,INSA,DUT'; mysql> INSERT INTO Cursus VALUES ('E3', 'Bug', ('BTS,INSA,ENAC')); ERROR 1265 (01000): Data truncated for column 'diplomes' at row 1</pre>

❖ *Dates et heures*

Dans cette section, on va étudier la manipulation des éléments de type date/heure. Nous verrons que MySQL peut les considérer soit en tant que chaînes de caractères soit comme numériques.

Formats

Concernant les types DATETIME, DATE et TIMESTAMP les formats possibles sont les suivants :

_ Chaînes de caractères 'YYYY-MM-DD HH:MM:SS' ou 'YY-MM-DD HH:MM:SS' (pour les colonnes DATE 'YYYY-MM-DD' ou 'YY-MM-DD'). Tout autre délimiteur est autorisé comme :

'2005.12.31 11%30%45' (pour les colonnes DATE, '65.12.31', '65/12/31', et '65@12@31' sont équivalents et désignent tous le réveillon de l'année 1965).

_ Considérés comme chaînes de caractères dans les formats suivants : 'YYYYMMDDHHMMSS' ou 'YYMMDDHHMMSS' (pour les DATE 'YYYYMMDD' ou 'YYMMDD') en supposant que la chaîne ait un sens en tant que date. Ainsi '19650205063000' est interprété comme le 5 février 1965 à 6 heures et 30 minutes. Par contre '19650255' n'a pas de sens du fait du jour (il sera interprété comme '0000-00-00').

_ Considérés comme numériques dans les formats suivants : YYYYMMDDHHMMSS ou YYMMDDHHMMSS (pour les DATE YYYYMMDD ou YYMMDD) en supposant que le nombre ait un sens en tant que date. Ainsi 19650205063000 est interprété comme le 5 février 1965 à 6 heures et 30 minutes. Par contre 19650205069000 n'a pas de sens du fait des minutes (il sera interprété comme 000000000000000).

Exemple avec DATE et DATETIME

Déclarons la table Pilote qui contient deux colonnes de type date/heure : DATE et DATETIME :

```
CREATE TABLE Pilote
(brevet VARCHAR(6), nom VARCHAR(20), dateNaiss DATE,
nbHVol DECIMAL(7,2), dateEmbauche DATETIME, compa VARCHAR(4),
CONSTRAINT pk_Pilote PRIMARY KEY(brevet));
```

L'insertion du pilote initialise la date de naissance au 5 février 1965, ainsi que celle de l'embauche à la date du moment (heures, minutes, secondes) par la fonction SYSDATE.

```
INSERT INTO Pilote  
VALUES ('PL-1', 'Christian Soutou', '1965-02-05', 900, SYSDATE(), 'AF');
```

Nous verrons au chapitre 4 comment extraire les années, mois, jours, heures, minutes et secondes. Nous verrons aussi qu'il est possible d'ajouter ou de soustraire des dates entre elles.

Exemple avec TIME et YEAR

Par analogie aux différents formats des dates, les heures (type TIME 'HH:MM:SS' ou 'HHH:MM:SS') peuvent aussi être manipulées sous la forme de chaînes ou de nombres.

_ Chaîne 'HH:MM:SS', 'HH:MM'.

_ Chaîne sans les délimiteurs sous réserve que la chaîne ait un sens. Ainsi, '101112' est considéré comme '10:11:12', mais '109712' est incorrect du fait des minutes, il devient '00:00:00'.

_ Nombre en raisonnant comme les chaînes. Ainsi 101112 est considéré comme '10:11:12', mais 109712 est incorrect du fait des minutes, il devient '00:00:00'.

Les formats suivants sont aussi corrects : SS, MMSS, HHMMSS.

La table Pilote suivante contient la colonne pasVolDepuis pour stocker le délai depuis le dernier vol et la colonne retraite qui indique l'année de retraite.

```
CREATE TABLE Pilote  
(brevet VARCHAR(6), nom VARCHAR(20), pasVolDepuis TIME, retraite YEAR,  
CONSTRAINT pk_Pilote PRIMARY KEY(brevet));
```

Les insertions suivantes utilisent différents formats. Le premier pilote n'a pas volé depuis 1 jour et 23 heures (47 heures) ; le second depuis 15 heures, 26 minutes 30 secondes ; le troisième depuis 4 jours et 23 heures (119 heures) ; le quatrième depuis 3 heures 27 minutes et 50 secondes, et le dernier se croit plus précis que tous les autres en ajoutant une fraction qui ne sera pas prise en compte au niveau de la base.

```

INSERT INTO Pilote VALUES ('PL-1', 'Hait', '1 23:0:0', '2002');
INSERT INTO Pilote VALUES ('PL-2', 'Crampes', '15:26:30', 2006);
INSERT INTO Pilote VALUES ('PL-3', 'Tuffery', '4 23:00', 05);
INSERT INTO Pilote VALUES ('PL-4', 'Mercier', '03:27:50', '07');
INSERT INTO Pilote VALUES ('PL-5', 'Albaric', '1 23:0:0.457', '01');

```

L'état de la base est le suivant :

```
mysql> SELECT * FROM Pilote;
```

```

+-----+-----+-----+-----+
| brevet | nom   | pasVolDepuis | retraite |
+-----+-----+-----+-----+
| PL-1   | Hait  | 47:00:00     | 2002     |
| PL-2   | Crampes | 15:26:30     | 2006     |
| PL-3   | Tuffery | 119:00:00    | 2005     |
| PL-4   | Mercier | 03:27:50     | 2007     |
| PL-5   | Albaric | 47:00:00     | 2001     |
+-----+-----+-----+-----+

```

Fonctions utiles

Les fonctions `CURRENT_TIMESTAMP()`, `CURRENT_DATE()` et `CURRENT_TIME()`, `UTC_TIME()`, renseignent sur l'instant, la date, l'heure et l'heure GMT de la session en cours.

Il n'est pas nécessaire d'utiliser une table pour afficher une expression dans l'interface de commande. L'exemple suivant montre que la requête a été exécutée le 1er novembre 2005 à 10 heures, 11 minutes et 27 secondes. Le client est sur le fuseau GMT+1h.

```
mysql> SELECT CURRENT_TIMESTAMP(), CURRENT_TIME(),
CURRENT_DATE(), UTC_
TIME();
```

```

+-----+-----+-----+-----+
| CURRENT_TIMESTAMP() | CURRENT_TIME() | CURRENT_DATE() | UTC_TIME() |
+-----+-----+-----+-----+
| 2005-11-01 10:11:27 | 10:11:27      | 2005-11-01     | 09:11:27   |
+-----+-----+-----+-----+

```

III. Modification de colonnes

L'instruction UPDATE permet la mise à jour des colonnes d'une table. Pour pouvoir modifier des enregistrements d'une table, il faut que cette dernière soit dans votre base ou que vous ayez reçu le privilège UPDATE sur la table.

Syntaxe (**UPDATE**)

La syntaxe simplifiée de l'instruction UPDATE est la suivante :

```
UPDATE [nomBase.] nomTable  
SET col_name1=expr1 [, col_name2=expr2 ...]  
SET colonne1 = expression1 | (requête_SELECT) | DEFAULT  
[,colonne2 = expression2...]  
[WHERE (condition)]  
[ORDER BY listeColonnes]  
[LIMIT nbreLimite]
```

_ La clause **SET** actualise une colonne en lui affectant une expression (valeur, valeur par défaut, calcul ou résultat d'une requête).

_ La condition du **WHERE** filtre les lignes à mettre à jour dans la table. Si aucune condition n'est précisée, tous les enregistrements seront actualisés. Si la condition ne filtre aucune ligne, aucune mise à jour ne sera réalisée.

_ **ORDER BY** indique l'ordre de modification des colonnes.

_ **LIMIT** spécifie le nombre maximum d'enregistrements à changer (par ordre de clé primaire croissante).

1. Modification d'une colonne

Modifions la compagnie de code 'AN1' en affectant la valeur 50 à la colonne nrue.

```
UPDATE Compagnie SET nrue = 50 WHERE comp = 'AN1';
```

2. Modification de plusieurs colonnes

Modifions la compagnie de code 'AN2' en affectant simultanément la valeur 14 à la colonne nrue et la valeur par défaut ('Paris') à la colonne ville.

```
UPDATE Compagnie SET nrue = 14, ville = DEFAULT WHERE comp = 'AN2';
```

La table Compagnie contient à présent les données suivantes :

Figure 6 Table après les modifications

Compagnie		Modification 1		
comp	nrue	rue	ville	nomComp
SING	7	Camparols	Singapour	Singapore AL
AF	10	Gambetta	Paris	Air France
AN1	50	Hoche	Blagnac	Air Nul1
AC	8	Champs Elysées	Paris	Castanet Air
AN2	14	Foch	Paris	Air Nul2

Modifications 2

3. Modification de plusieurs enregistrements

Modifions les deux premières compagnies (par ordre de clé primaire, ici 'AC' et 'AF') en affectant la valeur 'Toulouse' à la colonne ville.

UPDATE Compagnie SET ville = 'Toulouse' **LIMIT 2**;

- **Ne pas respecter les contraintes**

Il faut, comme pour les insertions, respecter les contraintes qui existent au niveau des colonnes. Dans le cas inverse, une erreur est renvoyée (le nom de la contrainte apparaît) et la mise à jour n'est pas effectuée.

Tableau 8 Table, données et contraintes

Données

Table et contraintes

Figure 2-7 Données

Pilote

brevet	nom	nbHVol	compa
PL-1	Louise Ente	450	AF
PL-2	Jules Ente	900	AF
PL-3	Paul Soutou	1000	SING

```
CREATE TABLE Pilote
(brevet CHAR(6), nom CHAR(15) NOT NULL,
nbHVol DECIMAL(7,2), compa CHAR(4),
CONSTRAINT pk_Pilote
PRIMARY KEY(brevet),
CONSTRAINT ck_nbHVol
CHECK (nbHVol BETWEEN 0 AND 20000),
CONSTRAINT un_nom UNIQUE(nom),
CONSTRAINT fk_Pil_compa_Comp
FOREIGN KEY (compa)
REFERENCES Compagnie(comp));
```

À partir de la table Pilote, le tableau suivant décrit des modifications (certaines ne vérifient pas de contraintes). La mise à jour d'une clé étrangère est possible si elle n'est pas référencée par une clé primaire.

Tableau 9 Modifications

Vérifiant les contraintes (sauf une !)	Ne vérifiant pas les contraintes
<pre>--Modification d'une clé étrangère UPDATE Pilote SET compa = 'SING' WHERE brevet = 'PL-2'; -- Modification d'une clé primaire UPDATE Pilote SET brevet = 'PL3bis' WHERE brevet = 'PL-3'; --Passe outre la contrainte CHECK ! UPDATE Pilote SET nbHVol= 30000 WHERE brevet = 'PL-1';</pre>	<pre>mysql> UPDATE Pilote SET brevet='PL-2' WHERE brevet='PL-1'; ERROR 1062 (23000): Duplicate entry 'PL-2' for key 1 mysql> UPDATE Pilote SET nom = NULL WHERE brevet = 'PL-1'; ERROR 1263 (22004): Column set to default value; NULL supplied to NOT NULL column 'nom' at row 1 mysql> UPDATE Pilote SET nom='Paul Soutou' WHERE brevet = 'PL-1'; ERROR 1062 (23000): Duplicate entry 'Paul Soutou' for key 2 mysql> UPDATE Pilote SET compa='TOTO' WHERE brevet = 'PL-1'; ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('bdsoutou/pilote', CONSTRAINT 'fk_Pil_compa_Comp' FOREIGN KEY ('compa') REFERENCES 'compagnie' ('comp'))</pre>

Figure 2-8 Après modifications

Pilote

brevet	nom	nbHVol	compa
PL-1	Louise Ente	30000	AF
PL-2	Jules Ente	900	SING
PL3bis	Paul Soutou	1000	SING

4. Remplacement d'un enregistrement

L'instruction REPLACE consiste, comme son nom l'indique, à remplacer un enregistrement dans sa totalité (toutes ses colonnes). Il faut avoir les privilèges INSERT et DELETE sur la table. C'est selon la valeur de la clé primaire ou celle d'un index unique que l'enregistrement sera remplacé.

Si la table ne dispose pas d'une contrainte PRIMARY KEY ou UNIQUE, l'utilisation de **REPLACE**

n'a pas de sens et devient équivalente à INSERT.

La syntaxe simplifiée de l'instruction UPDATE est la suivante :

REPLACE [INTO] [nomBase.] nomTable [(colonne1,...)]

VALUES ({*expressionI* | DEFAULT},...) [(...),...]

_ LOW_PRIORITY et DELAYED ont la même signification que pour INSERT et UPDATE.

_ VALUES contient les valeurs de remplacement.

L'instruction suivante remplace l'enregistrement relatif à la compagnie de code 'AN1' :

REPLACE INTO Compagnie VALUES ('AN1', 24, 'Salas', 'Ramonville','Air RENATO');

IV. Suppression d'enregistrements

Les instructions **DELETE** et **TRUNCATE** permettent de supprimer un ou plusieurs enregistrements d'une table. Pour pouvoir supprimer des enregistrements dans une table, il faut que cette dernière soit dans votre base ou que vous ayez reçu le privilège DELETE sur la table.

Instruction **DELETE**

La syntaxe simplifiée de l'instruction DELETE est la suivante :

DELETE FROM [*nomBase.*] *nomTable*

[WHERE (*condition*)]

[ORDER BY *listeColonnes*]

[LIMIT *nbreLimite*]

_ La condition du **WHERE** sélectionne les lignes à supprimer dans la table. Si aucune condition n'est précisée, toutes les lignes seront détruites. Si la condition ne sélectionne aucune ligne, aucun enregistrement ne sera supprimé.

_ **ORDERBY** réalise un tri des enregistrements qui seront effacés dans cet ordre.

Détaillons les possibilités de cette instruction en considérant les différentes tables précédemment définies. La première commande supprime tous les pilotes de la compagnie de code 'AF', la seconde, avec une autre écriture, détruit la compagnie de code 'AF'.

DELETE FROM Pilote WHERE compa = 'AF';

DELETE FROM Compagnie WHERE comp = 'AF';

Détruisons enfin les deux premières compagnies (triées par ordre croissant de clé, ici 'AC' et 'AN1') qui ne sont référencées par aucun pilote.

DELETE FROM Compagnie **LIMIT 2**;

Instruction **TRUNCATE**

La commande TRUNCATE est une extension de SQL qui a été proposée par Oracle et reprise par MySQL. Cette commande supprime tous les enregistrements d'une table et libère éventuellement l'espace de stockage utilisé par la table. La syntaxe est la suivante :

TRUNCATE [TABLE] [*nomBase.*] *nomTable*;